

# 프로그래밍응용개념

#07 클래스 (접근제한자, 싱글톤 패턴)

국립한밭대학교 인공지능 소프트웨어학과

이성민

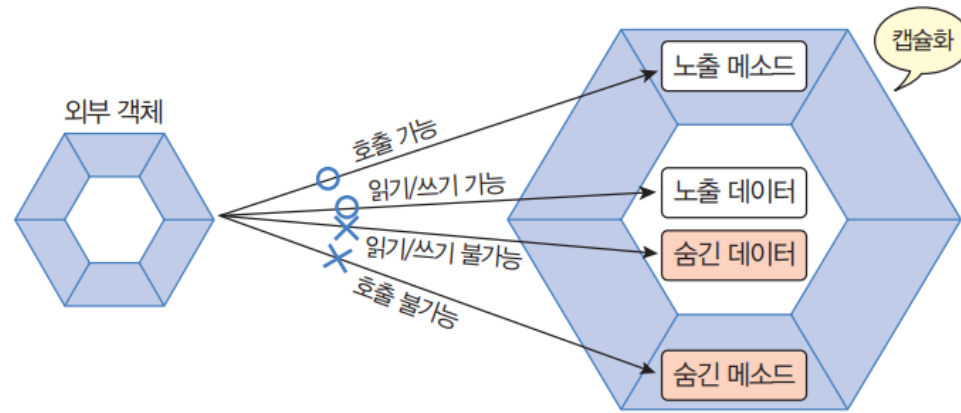
# 클래스

- 객체지향 프로그래밍
- 클래스
- 싱글톤 패턴

# 객체 지향 프로그래밍

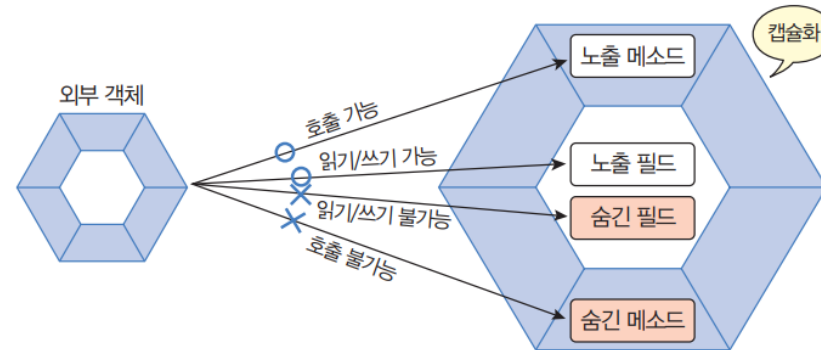
- 객체 지향 프로그래밍의 특징

- 캡슐화: 객체의 데이터 (필드), 동작 (메소드)을 하나로 묶고 실제 구현 내용을 외부에 감추는 것

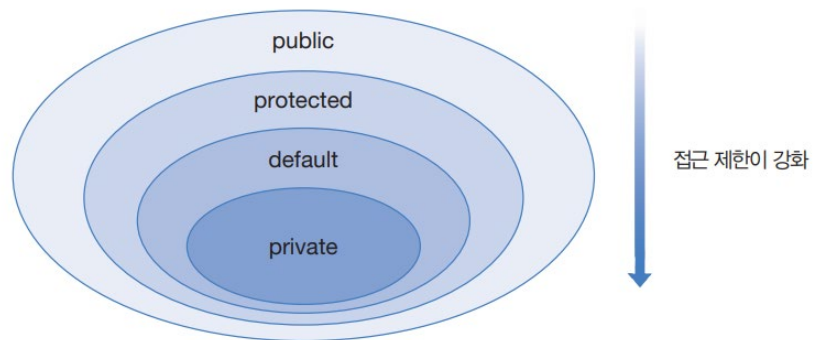


# 접근 제한자

- 접근 제한자
  - 중요한 필드와 메소드가 외부로 노출되지 않도록 함



- 접근 제한자: public, protected, private

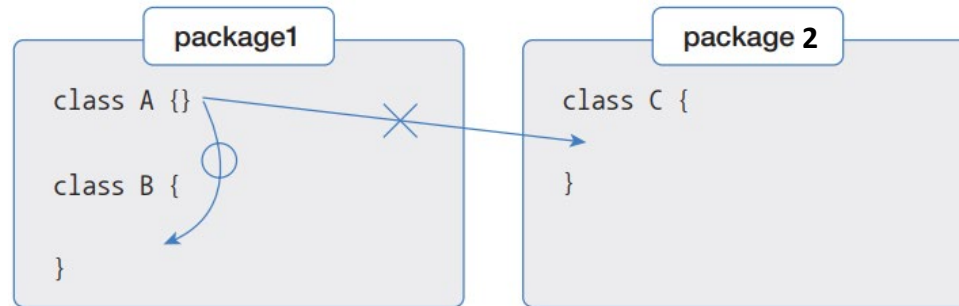


| 접근 제한자    | 제한 대상             | 제한 범위                                       |
|-----------|-------------------|---------------------------------------------|
| public    | 클래스, 필드, 생성자, 메소드 | 없음                                          |
| protected | 필드, 생성자, 메소드      | 같은 패키지이거나, 자식 객체만 사용 가능<br>(7장 상속에서 자세히 설명) |
| (default) | 클래스, 필드, 생성자, 메소드 | 같은 패키지                                      |
| private   | 필드, 생성자, 메소드      | 객체 내부                                       |

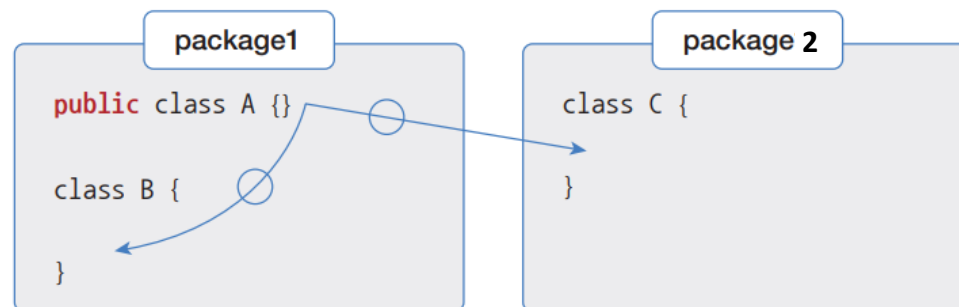
# 접근 제한자

- 클래스의 접근 제한

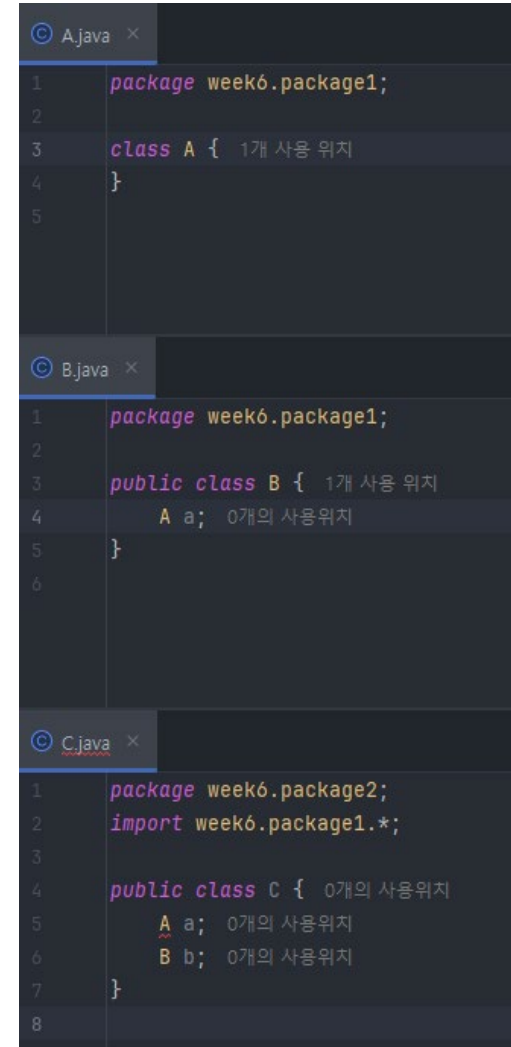
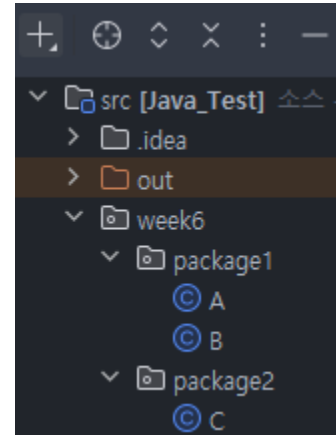
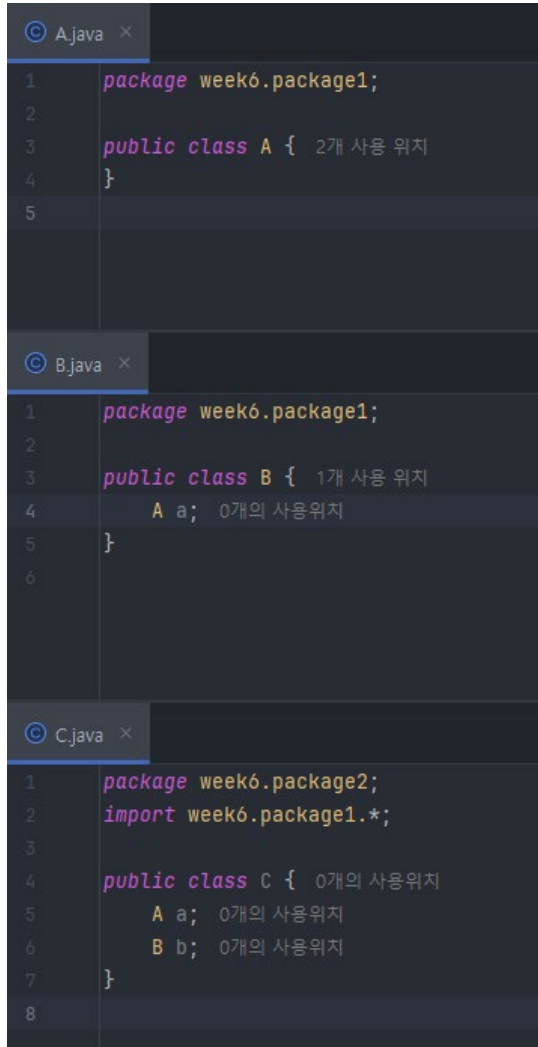
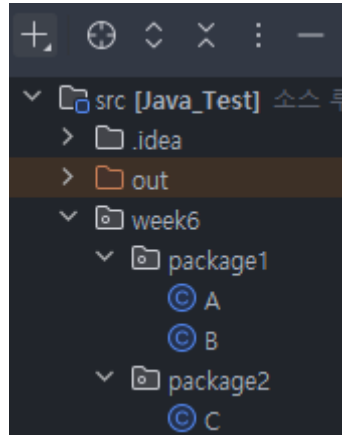
- public 접근 제한자를 생략하면, 다른 패키지에서 사용할 수 없음



- public 접근 제한자를 붙이면, 다른 패키지에서도 사용할 수 있음



# 접근 제한자



# 접근 제한자

- 생성자의 접근 제한

- 생성자는 public, default, private 접근 제한을 가질 수 있음

```
public class ClassName {  
    //생성자 선언  
    [ public | private ] ClassName(...) { ... }  
}
```

| 접근 제한자  | 생성자      | 설명                                                    |
|---------|----------|-------------------------------------------------------|
| public  | 클래스(...) | 모든 패키지에서 생성자를 호출할 수 있다.<br>= 모든 패키지에서 객체를 생성할 수 있다.   |
|         | 클래스(...) | 같은 패키지에서만 생성자를 호출할 수 있다.<br>= 같은 패키지에서만 객체를 생성할 수 있다. |
| private | 클래스(...) | 클래스 내부에서만 생성자를 호출할 수 있다.<br>= 클래스 내부에서만 객체를 생성할 수 있다. |

# 접근 제한자

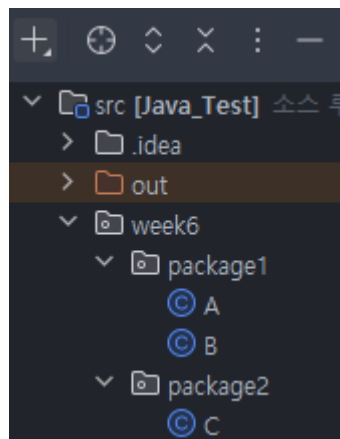
```

A.java
1 package week6.package1;
2
3 public class A {
4     A a1 = new A(b: true);
5     A a2 = new A(i: 1);
6     A a3 = new A(s: "string");
7
8     public A(boolean b) {}
9     A(int i) {}
10    private A(String s) {}
11 }

B.java
1 package week6.package1;
2
3 public class B {
4     A a1 = new A(b: false);
5     A a2 = new A(i: 3);
6     A a3 = new A("STRING");
7 }

C.java
1 package week6.package2;
2 import week6.package1.*;
3
4 public class C {
5     A a1 = new A(b: true);
6     A a2 = new A(b: 3);
7     A a3 = new A(b: "문자열");
8 }

```



| 접근 제한자    | 제한 대상             | 제한 범위                                       |
|-----------|-------------------|---------------------------------------------|
| public    | 클래스, 필드, 생성자, 메소드 | 없음                                          |
| protected | 필드, 생성자, 메소드      | 같은 패키지이거나, 자식 객체만 사용 가능<br>(7장 상속에서 자세히 설명) |
| (default) | 클래스, 필드, 생성자, 메소드 | 같은 패키지                                      |
| private   | 필드, 생성자, 메소드      | 객체 내부                                       |



# 접근 제한자

- 필드와 메소드의 접근 제한
  - 필드와 메소드는 public, default, private 접근 제한을 가질 수 있음

```
//필드 선언  
[ public | private ] 타입 필드;  
  
//메소드 선언  
[ public | private ] 리턴타입 메소드(...) { ... }
```

| 접근 제한자  | 생성자            | 설명                                                     |
|---------|----------------|--------------------------------------------------------|
| public  | 필드<br>메소드(...) | 모든 패키지에서 필드를 읽고 변경할 수 있다.<br>모든 패키지에서 메소드를 호출할 수 있다.   |
|         | 필드<br>메소드(...) | 같은 패키지에서만 필드를 읽고 변경할 수 있다.<br>같은 패키지에서만 메소드를 호출할 수 있다. |
| private | 필드<br>메소드(...) | 클래스 내부에서만 필드를 읽고 변경할 수 있다.<br>클래스 내부에서만 메소드를 호출할 수 있다. |

# 접근 제한자

```
A.java
1 package week6.package1;
2
3 public class A {
4     public int field1;
5     int field2;
6     private int field3;
7
8     public void method1() {}
9     void method2() {}
10    private void method3() {}
11
12    public A() {
13        field1 = 1;
14        field2 = 2;
15        field3 = 3;
16        method1();
17        method2();
18        method3();
19    }
20 }
21

B.java
1 package week6.package1;
2
3 public class B {
4     public B() {
5         A a = new A();
6         a.field1 = 1;
7         a.field2 = 2;
8         a.field3 = 3;
9
10        a.method1();
11        a.method2();
12        a.method3();
13    }
14 }
15

C.java
1 package week6.package2;
2 import week6.package1.*;
3
4 public class C {
5     public C() {
6         A a = new A();
7         a.field1 = 1;
8         a.field2 = 2;
9         a.field3 = 3;
10
11        a.method1();
12        a.method2();
13        a.method3();
14    }
15 }
16 }
17
```

# Getter와 Setter

- Getter
  - 외부에서 직접적인 필드 접근을 막고, 메소드를 통해 필드에 접근
- Setter
  - 데이터를 검증해서 유효한 값만 필드에 저장하는 메소드

```
private 타입 fieldName;
```

필드 접근 제한자: private

```
//Getter
```

```
public 타입 getFieldName() {  
    return fieldName;  
}
```

접근 제한자: public

리턴 타입: 필드타입

메소드 이름: get + 필드이름(첫 글자 대문자)

리턴값: 필드값

```
//Setter
```

```
public void setFieldName(타입 fieldName) {  
    this.fieldName = fieldName;  
}
```

접근 제한자: public

리턴 타입: void

메소드 이름: set + 필드이름(첫 글자 대문자)

매개변수 타입: 필드타입

# Getter와 Setter

```
Main.java x
1 public class Main {
2     public static void main(String[] args) {
3         BankAccount bankAccount = new BankAccount( accountNumber: "123456",
4                                                     owenerName: "김한밭",
5                                                     password: "1234");
6
7         String accountNumber = bankAccount.getAccountNumber();
8         System.out.println(accountNumber);
9
10        String ownerName = bankAccount.getOwnerName();
11        System.out.println(ownerName);
12
13        bankAccount.setPassword("asdf");
14        String password = bankAccount.getPassword();
15        System.out.println(password);
16
17        String accountNumber1 = bankAccount.accountNumber; // X
18        String ownerName1 = bankAccount.ownerName; // X
19        String password1 = bankAccount.password; // X
20        bankAccount.password = "asdf"; // X
21    }
22 }
23
24
25
26
27
28

BankAccount.java x
1 public class BankAccount { 2개 사용 위치
2     private String accountNumber; 2개 사용 위치
3     private String ownerName; 2개 사용 위치
4     private String password; 3개 사용 위치
5
6     public BankAccount(String accountNumber, String owenerName, String password) {
7         this.accountNumber = accountNumber;
8         this.ownerName = ownerName;
9         this.password = password;
10    }
11
12    public String getAccountNumber() { 1개 사용 위치
13        return this.accountNumber;
14    }
15
16    public String getOwnerName() { 1개 사용 위치
17        return this.ownerName;
18    }
19
20    public String getPassword() { 1개 사용 위치
21        return this.password;
22    }
23
24    public void setPassword(String newPass) { 1개 사용 위치
25        this.password = newPass;
26    }
27
28 }
```

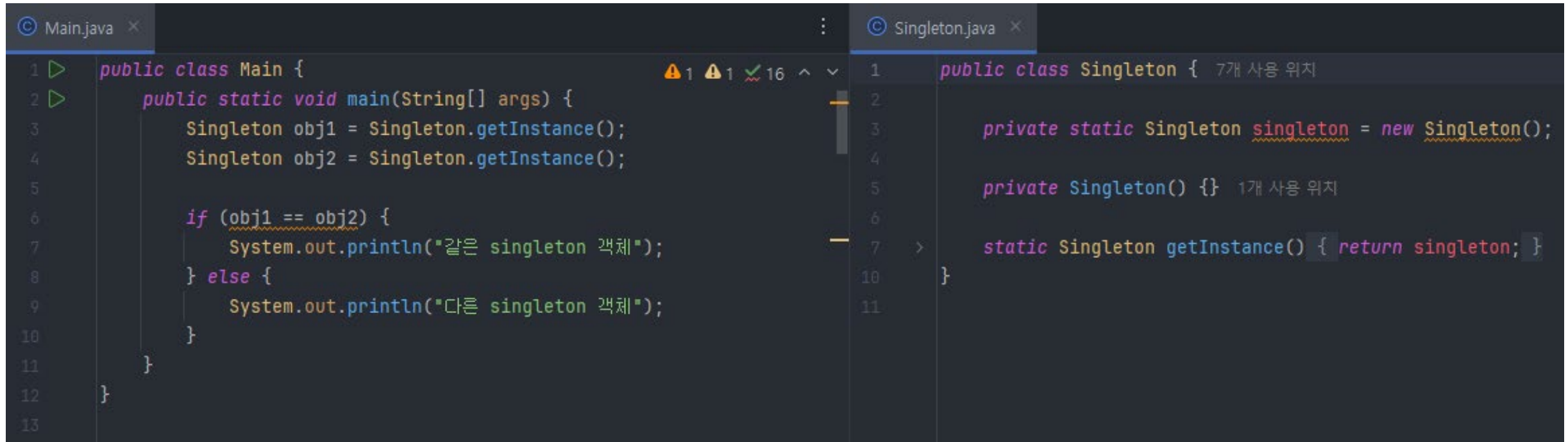
# 싱글톤 패턴

- 싱글톤 패턴

- 단 한 개의 객체만 생성해서 사용하고 싶은 경우, 생성자를 private 접근 제한해서 외부에서 new 연산자로 생성자를 호출할 수 없도록 막음
- 대신, 싱글톤 패턴이 제공하는 정적 메소드를 통해 간접적으로 객체 접근

```
public class 클래스 {  
    //private 접근 권한을 갖는 정적 필드 선언과 초기화  
    private static 클래스 singleton = new 클래스(); •----- ①  
  
    //private 접근 권한을 갖는 생성자 선언  
    private 클래스() {}  
  
    //public 접근 권한을 갖는 정적 메소드 선언  
    public static 클래스 getInstance() { •----- ②  
        return singleton;  
    }  
}
```

# 싱글톤 패턴



```
1 public class Main {
2     public static void main(String[] args) {
3         Singleton obj1 = Singleton.getInstance();
4         Singleton obj2 = Singleton.getInstance();
5
6         if (obj1 == obj2) {
7             System.out.println("같은 singleton 객체");
8         } else {
9             System.out.println("다른 singleton 객체");
10        }
11    }
12 }
13
```

```
1 public class Singleton { 7개 사용 위치
2
3     private static Singleton singleton = new Singleton();
4
5     private Singleton() {} 1개 사용 위치
6
7     static Singleton getInstance() { return singleton; }
10 }
11
```



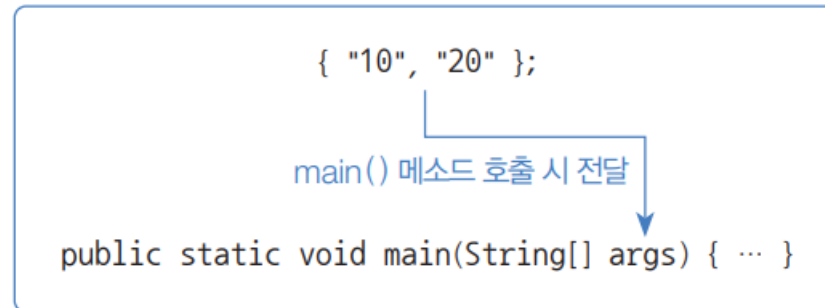
# 싱글톤 패턴

```
Main.java x
1 public class Main {
2     public static void main(String[] args) {
3         PrinterManager printer1 = PrinterManager.getInstance();
4         PrinterManager printer2 = PrinterManager.getInstance();
5         PrinterManager printer3 = PrinterManager.getInstance();
6         printer1.requestPrint( documentName: "기계학습.pdf");
7         printer2.requestPrint( documentName: "딥러닝.pdf");
8         printer3.requestPrint( documentName: "자바.pdf");
9
10        printer1.printAllJobs();
11    }
12 }
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27

PrinterManager.java x
1 public class PrinterManager { 9개 사용 위치
2     private String[] printQueue; 3개 사용 위치
3     private int printCount; 4개 사용 위치
4     private static PrinterManager printerManager = new PrinterManager();
5
6
7     private PrinterManager() { 1개 사용 위치
8         this.printQueue = new String[50];
9         this.printCount = 0;
10    }
11
12    static PrinterManager getInstance() { 3개 사용 위치
13        return printerManager;
14    }
15
16    public void requestPrint(String documentName) { 3개 사용 위치
17        this.printQueue[printCount] = documentName;
18        printCount++;
19    }
20
21    public void printAllJobs() { 1개 사용 위치
22        System.out.println("---현재 출력 대기 문서---");
23        for (int i=0; i<printCount; i++) {
24            System.out.println(printQueue[i]);
25        }
26    }
27 }
```

# 메인 메소드

- String[] args 매개변수
  - 자바 프로그램을 실행하기 위한 main() 메소드에는 문자열 배열인 String[] args 매개변수가 필요





# 메인 메소드

- String[] args 매개변수

```
public static void main(String[] args) {  
    if(args.length != 2) {  
        System.out.println("프로그램 입력 값이 부족");  
        System.exit(status: 0);  
    }  
  
    String strNum1 = args[0];  
    String strNum2 = args[1];  
  
    int num1 = Integer.parseInt(strNum1);  
    int num2 = Integer.parseInt(strNum2);  
  
    int result = num1 + num2;  
    System.out.println(num1 + " + " + num2 + " = " + result);  
}
```

```
(base) PS D:\Workspace\ThisIsJava> java .\src\ch05\sec11\MainStringArrayArgument.java 1 3  
1 + 3 = 4
```

# 메인 메소드

- String[] args 매개변수

